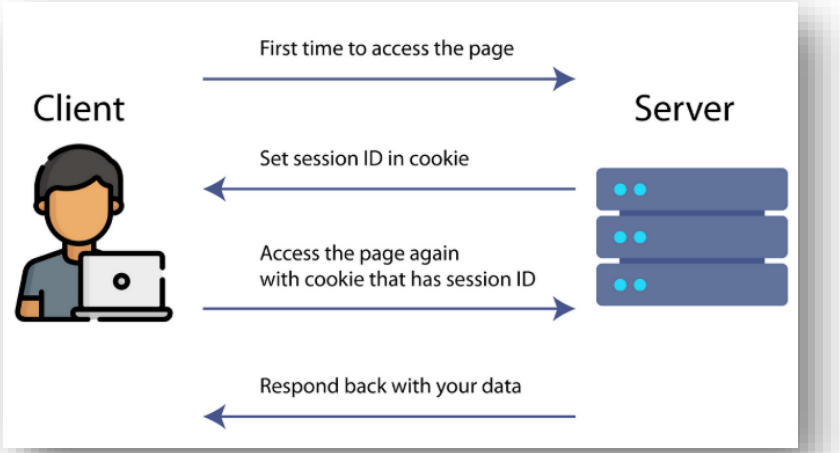
Høgskolen på Vestlandet



DAT108
Programmering og webapplikasjoner

F19 –PRG Sesjoner Scopes Innlogging

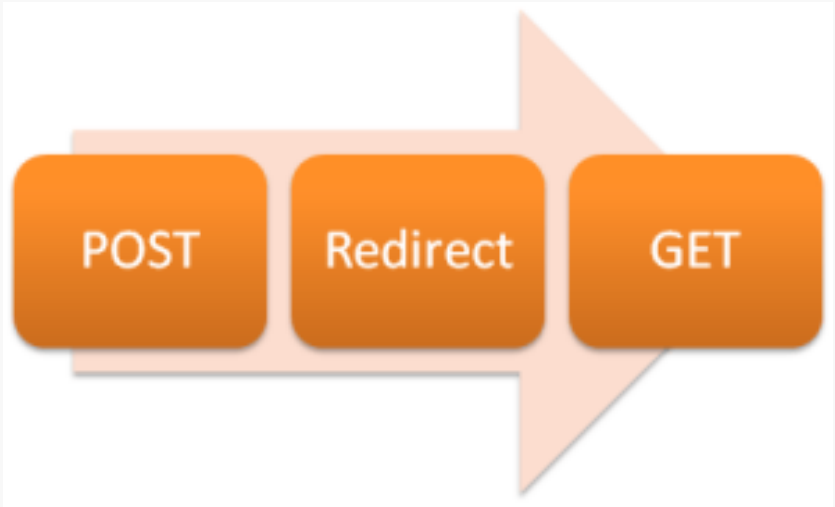
Lars-Petter Helland, 19.10.2026

I dag

- POST, dobbeltposting, redirect, PRG
- Sesjoner og sesjonsdata
- Data- /attributt-scopes
- Innlogging, autentisering og utlogging
- Et litt "større" eksempel

Dagens lesestoff

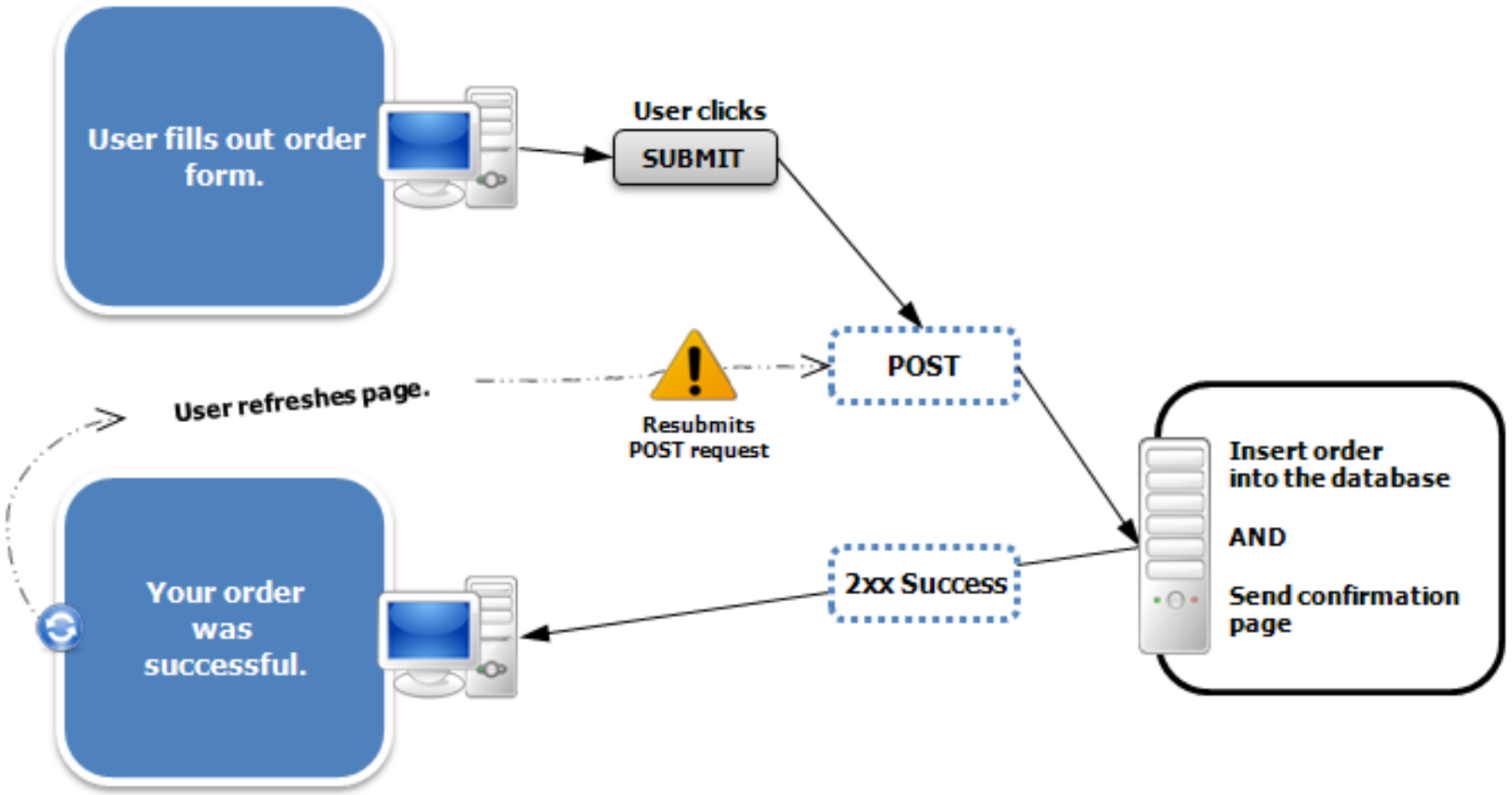
- ...

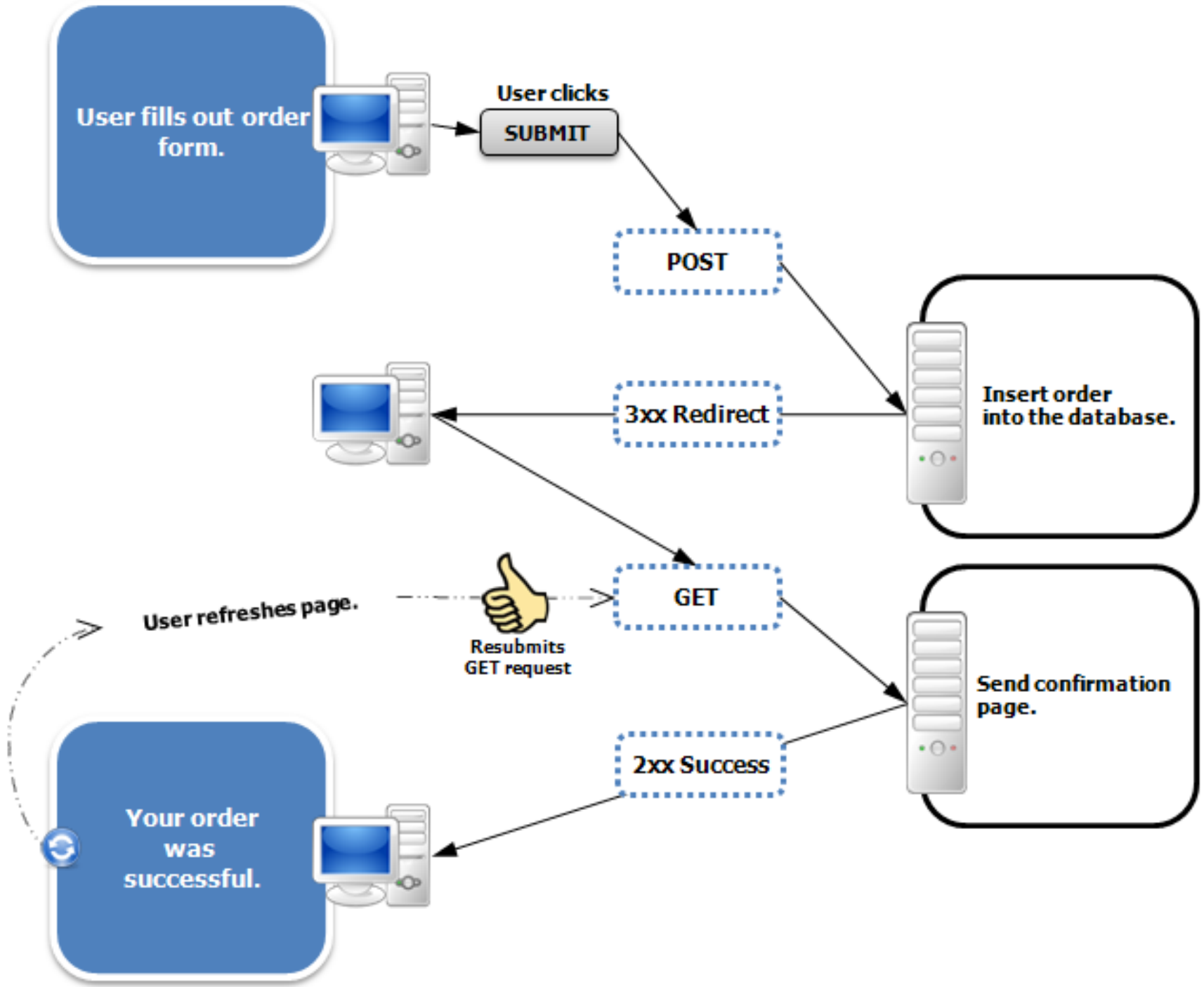

POST
Redirect
GET

Post-Redirect-Get

- POST-forespørsler brukes i hovedsak til å sende data, f.eks. logge inn, bestille billett, melde på arrangement, osv…
- Etter at dataene er mottatt og behandlet, ønsker vi å presentere en ny side for brukeren, f.eks. en kvittering.
- Hvis dette gjøres som en del av POST-requesten vil en refresh (gjenta forespørselen) i nettlseseren av kvitteringssiden føre til dobbeltposting. Det er heller ikke mulig å lage en kvittering med unik id (URL) som kan lagres for senere fremhenting.

- En løsningen er at vi etter en POST videresender nettleseren til en annen side (med GET).
- Dette utformingsmønsteret kalles **"Post Redirect Get"** (**PRG**).


User fills out order form.
User clicks
SUBMIT
POST
Resubmits
POST request
User refreshes page.
Insert order into the database
AND
Send confirmation page
2xx Success
Your order was successful.

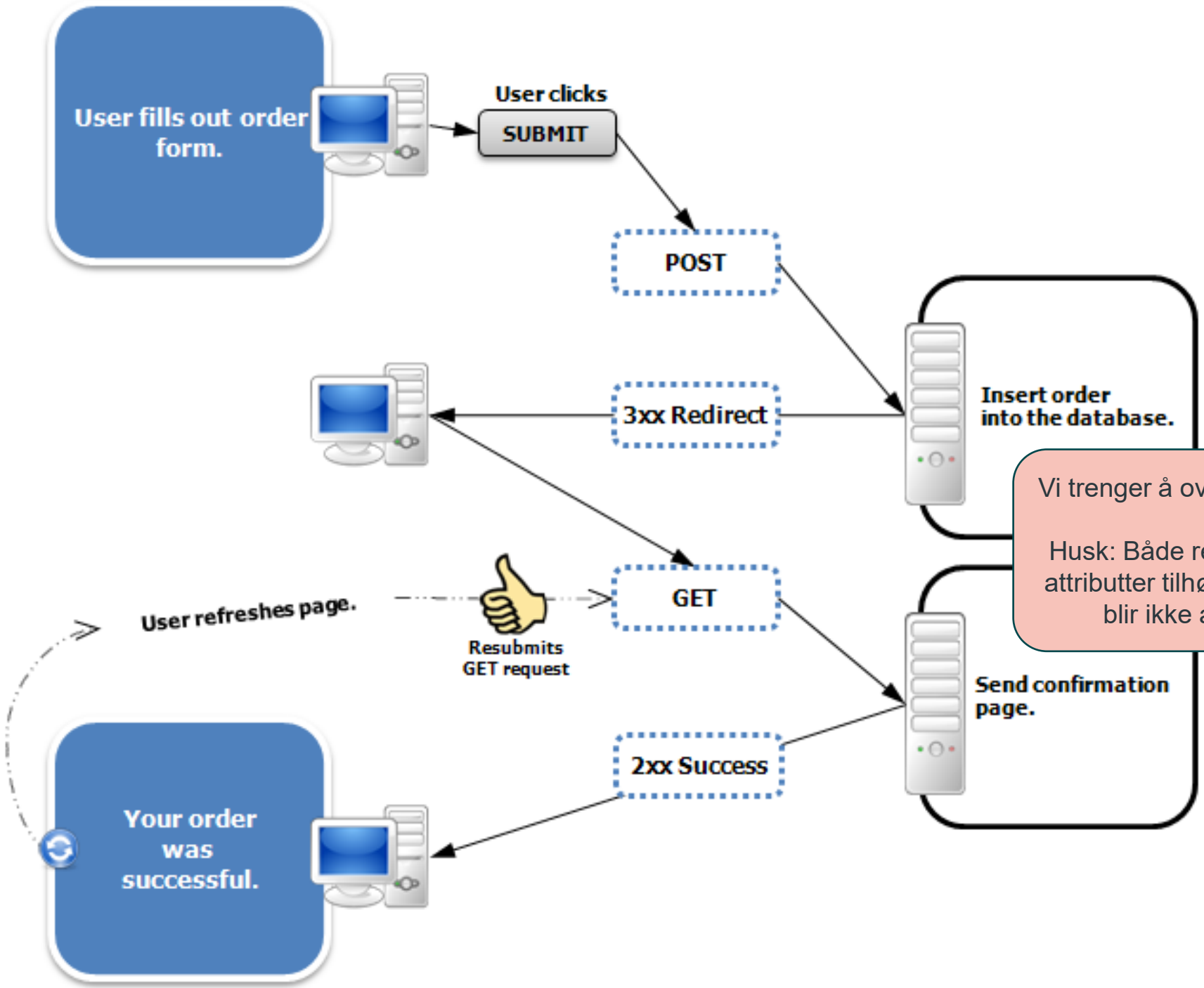

User fills out order form.
User clicks
SUBMIT
POST
3xx Redirect
Insert order into the database.
User refreshes page.
Resubmits GET request
GET
Send confirmation page.
2xx Success
Your order was successful.

PRG - Kode

```
@PostMapping("/bestilling")
public String bestillMedPrg(String vare, ...) {

        // NB! Før vi gjør en redirect, som fører til en ny request,
        // må vi finne en måte å gjøre String vare tilgjengelig etter
        // redirect. Vi skal se på noen alternativer.

        return "redirect:kvittering"; //NB! kvittering er en URL, ikke et view.
}
```



Vi trenger å overføre data til neste request !

Husk: Både request-parametre og model-attributter tilhører den første requesten, og blir ikke automatisk med videre !

PRG - Kode

```
@PostMapping("/bestilling")
public String bestillMedPrg(String vare, RedirectAttributes ra) {

        // Alternativ 1 (lagre parameteren i en redirect attribute)
        ra.addAttribute("vare", vare);
        // JEPP. Denne vil gjøres om til en parameter i neste request.
        // Fordel: Bokmerking.
        // Ulempe: Attributt må kunne kodes som en String

        return "redirect:kvittering"; //NB! kvittering er en URL, ikke et view.
}
```

PRG - Kode

```
@PostMapping("/bestilling")
public String bestillMedPrg(String vare, RedirectAttributes ra) {

        // Alternativ 2 (lagre parameteren i en redirect FLASH attribute)
        ra.addFlashAttribute("vare", vare);
        // JEPP. Denne vil lagres på server og knyttes til model i neste request.
        // Fordel: Greiest hvis det kun er snakk om å overføre data til neste request.
        // Ulempe: Blir ikke en del av URL. Unikt bokmerke må lages utenom.

        return "redirect:kvittering"; //NB! kvittering er en URL, ikke et view.
}
```

Eksempel – Unngå dobbeltposting med PRG

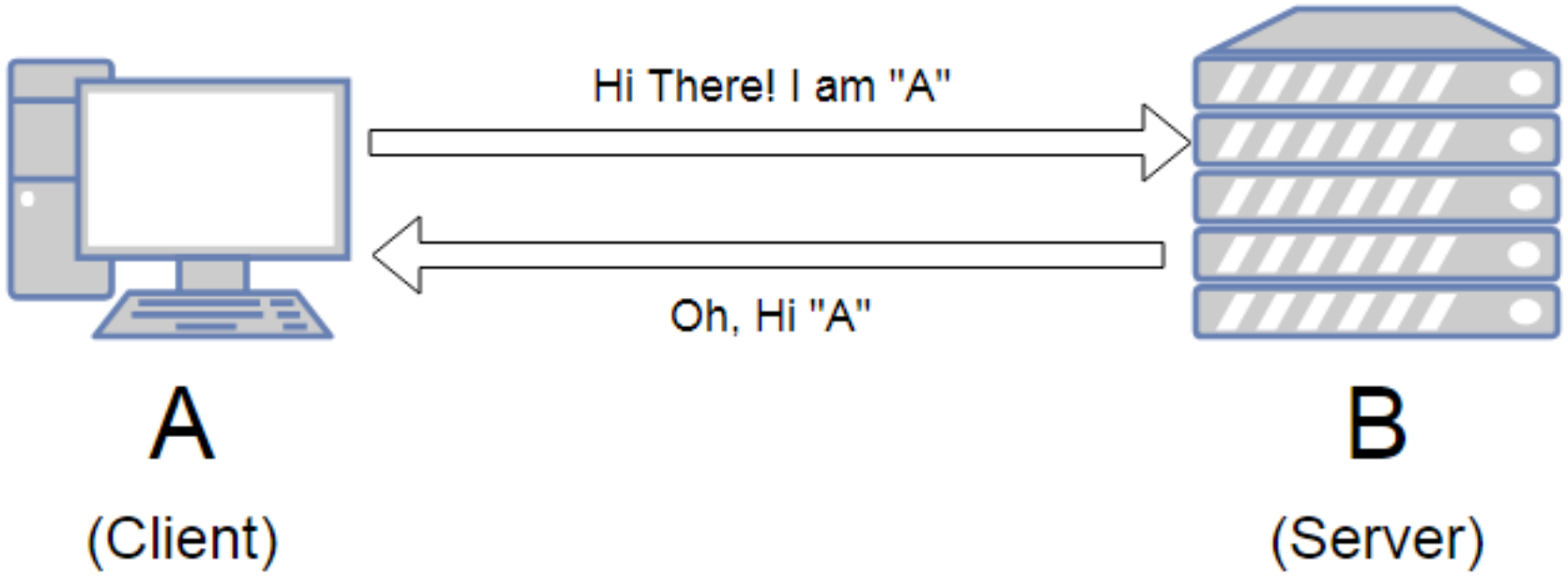
Prosjekt: 📂 f19a-post-redirect-get

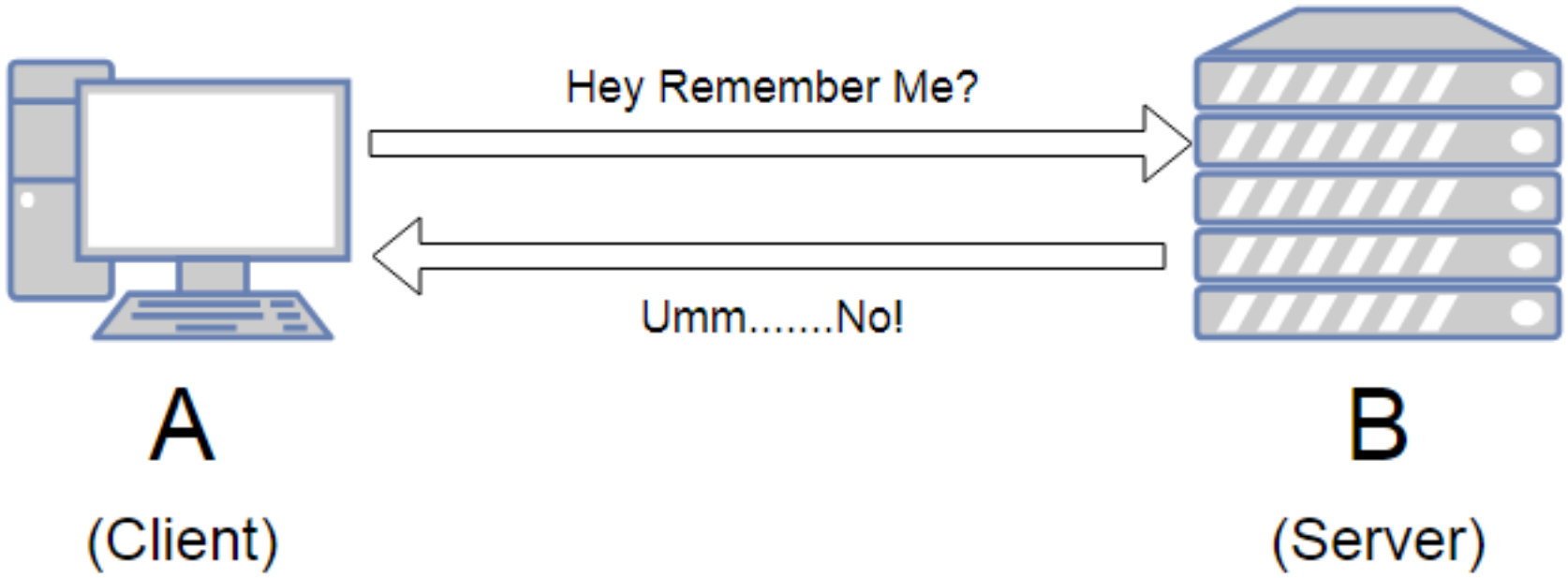


Sesjoner



https://medium.com/@maheshlsingh8412/cookie-session-story-of-a-stateless-http-3cd09cc01541




Hey Remember Me?
Umm.....No!
A
(Client)
B
(Server)

HTTP er tilstandsløs

- Nettleseren må enten, direkte eller indirekte, sende ALL nødvendig informasjon til tjeneren HVER gang!
- For hver eneste forespørsel må tjeneren finne ut:
  - Hvem er brukeren?
  - Har brukeren rettigheter til å utføre forespørselen?
  - Hvor langt var vi kommet?
  - Hvilke data har vi og hvilke mangler vi?

Et par vanlige scenarier

- Handlevogn:
  - Hvem er brukeren?
  - Har denne brukeren en handlevogn?
  - Hvilke varer er i denne brukerens handlevogn?

- Tjeneste for innlogget bruker:
  - Hvem er brukeren?
  - Er denne brukeren innlogget?
  - Hvilke data har vi på denne brukeren?

Hvilke data trenger vi å sende?

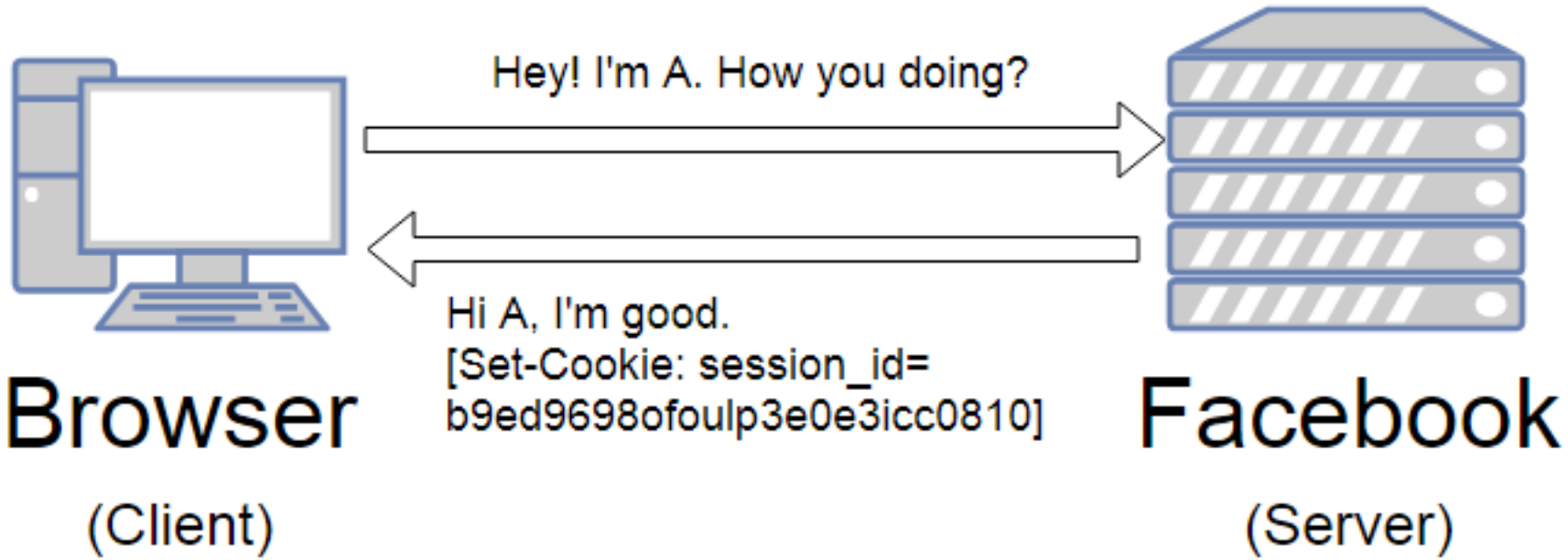
- Hvis vi ikke ønsker å lagre informasjon på tjeneren mellom forespørsler må ALL info sendes fra nettleseren HVER gang. Dette kan gjøres i URL, i (hidden) input-parametre og i **cookies**.

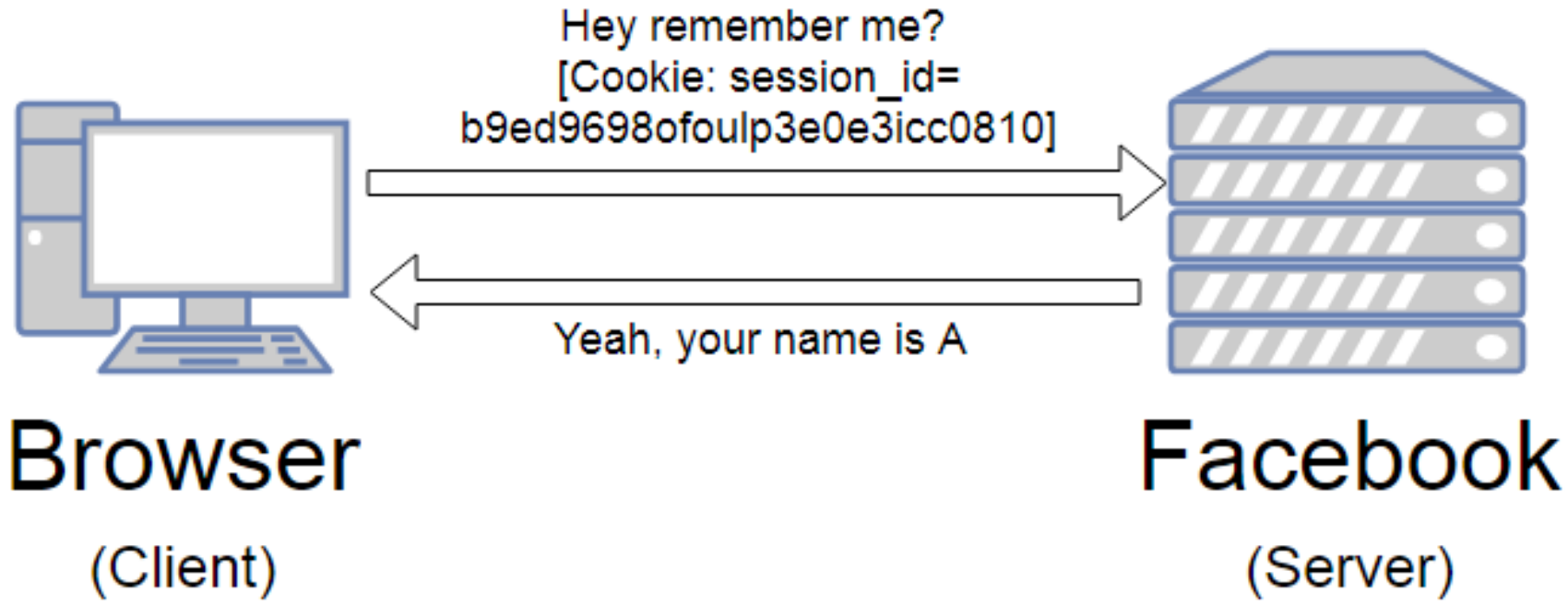
- Vanligvis **kan** tjeneren lagre informasjon. Vi kan da i stedet nøye oss med å sende en **sesjons-id** (for denne nettleser-sesjonen) som da knyttes til data lagret på tjeneren.

Sesjons-id og cookies

- Det mest vanlige (og det vi skal gjøre) er å ha en unik id knyttet til nettleser-økten / -sesjonen.
- Den mest vanlige måten er å sende en slik til klienten i responsen, og at klienten sende denne tilbake ved hver senere forespørsel. Dette er en **cookie**.
- En **cookie** er informasjon (en string) som nettleseren lagrer, og som sendes med ved senere requests til samme nettsted.

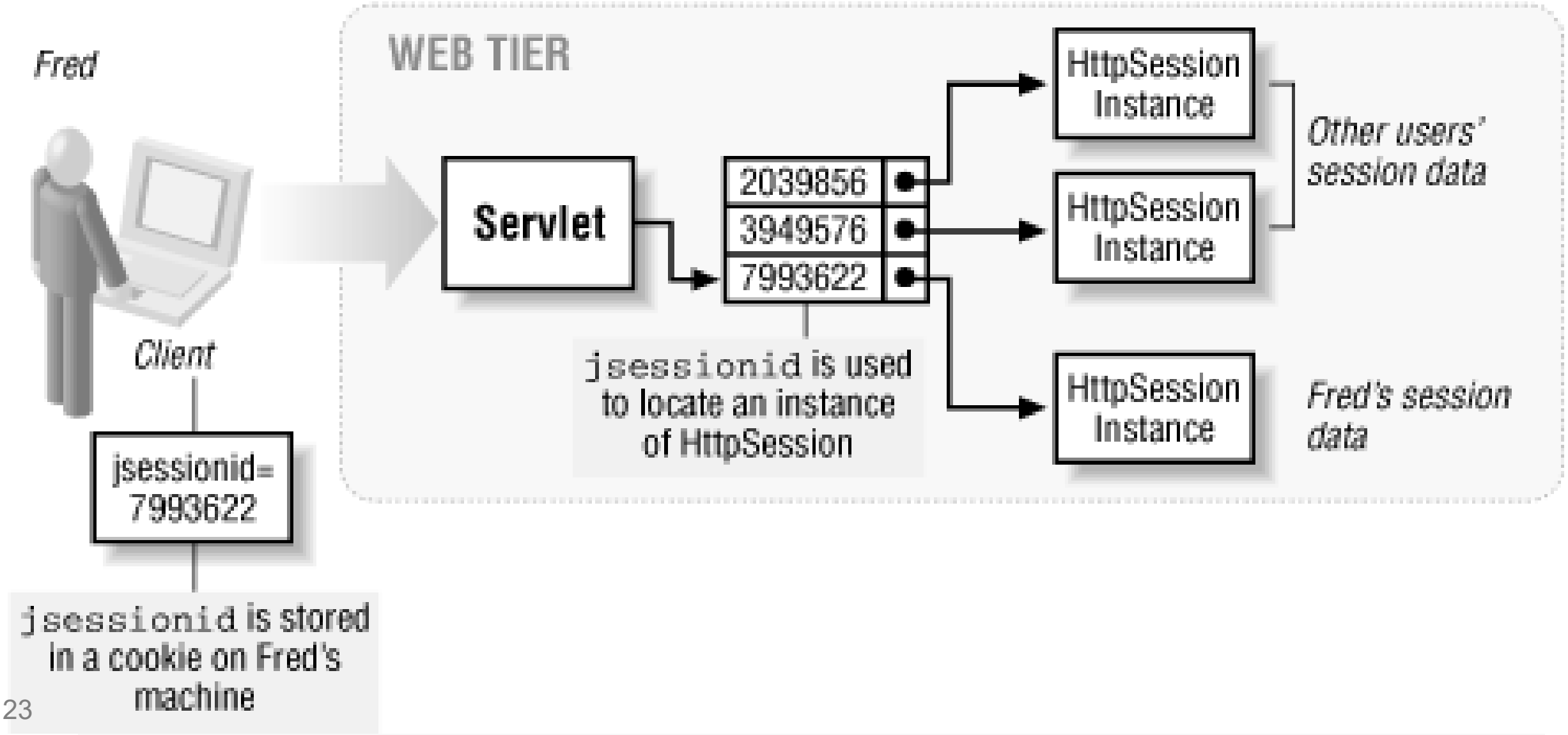
En alternativ sendingsmåte innebærer såkalt URL-omskriving. Id-en "bakes" da inn i alle URL-er til denne applikasjonen. Dette kan være en plan B om nettleseren har slått av cookies.


Hey! I'm A. How you doing?
Hi A, I'm good.
[Set-Cookie: session_id=
b9ed9698ofoulp3e0e3icc0810]
Browser
(Client)
Facebook
(Server)


Hey remember me?
[Cookie: session_id=
b9ed9698ofoulp3e0e3icc0810]
Yeah, your name is A
Browser
(Client)
Facebook
(Server)

Sesjonsdata

- Vi kan knytte data på tjeneren til en bestemt sesjons-id. Disse dataene deles ikke på tvers av brukere / klienter, men tilhører en bestemt bruker-økt.
- I Java web-applikasjoner er dette lagt opp som et Map av "sesjonsobjekter", der hvert sesjonsobjekt blant annet inneholder et Map av "attributter".


Fred
WEB TIER
Client
jsessionid=
7993622
jsessionid is stored
in a cookie on Fred's
machine
Servlet
2039856
3949576
7993622
jsessionid is used
to locate an instance
of HttpSession
HttpSession
Instance
HttpSession
Instance
HttpSession
Instance
Other users'
session data
Fred's session
data

Java `HttpSession` vs. Spring Sessions vs. SessionScoped Spring Components

Spring-rammeverket er et rammeverk på toppen av Java sitt standard web-rammeverk (.. Servlet-API). Tanken bak Spring er generelt både å **forbedre** og **forenkle**.

Spring MVC gir oss valget mellom å bruke Spring sine forbedre/forenklede måter å gjøre ting, eller å bruke Java sitt Servlet-API direkte i Spring-applikasjoner.

I DAT108 (siden det er et intro-kurs) er fokuset å bruke Spring til å **forenkle**.

Session-håndtering i Spring er en forbedring, men ikke bare en forenkling. Vi vil derfor benytte Java sin standard håndtering av sesjoner via **HttpRequest** og **HttpSession** i DAT108.

(Hvis du er interessert, kan du lese mer om Spring Session f.eks. her: https://www.baeldung.com/spring-session)

`HttpSession` API

Som tidligere nevnt kan en request sende med en **cookie** kalt **JSESSIONID** som den i forveien har fått av serveren (i en tidligere respons via headeren Set-Cookie).

SessionId-en (altså verdien til cookien JSESSIONID), f.eks. **5C5AF6A3B0B4694FD2865F317EBEB218**, vil på serveren brukes som nøkkel ved oppslag til data tilhørende denne sesjonen.

F.eks. vil hver sesjon ha et Map av attributter (på samme måte som request/model har et Map av attributter).

`HttpSession` API

I Controlleren får vi tilgang til sesjonsdata via en metode-parameter av typen **HttpSession**.

Alternativt indirekte via en metode-parameter av typen **HttpServletRequest**. Sistnevnte gir oss også mulighet til å opprette og invalidere en sesjon på serveren, noe som er viktig i forbindelse med inn- og utlogging:

```
@GetMapping public String ...(.., HttpServletRequest request) {
```

Å få tak i et eksisterende gyldig sesjonsobjekt (evt. null om det ikke finnes noen):

```
HttpSession sesjon = request.getSession(false);
```

Å opprette et sesjonsobjekt (evt. få et eksisterende gyldig hvis det finnes):

```
HttpSession sesjon = request.getSession(true);
```

Å gjøre ugyldig (invalidere) et sesjonsobjekt:

```
sesjon.invalidate();
```

`HttpSession` API forts.

Ta vare på data knyttet til sesjonen:

```
sesjon.setAttribute(<navn>, <objekt>);
```

Hente frem data knyttet til sesjonen:

```
T data = (T) sesjon.getAttribute(<navn>);
```

Data vi tar vare på på denne måten "lever forbi" requesten, og er tilgjengelig fra alle controllere/views i applikasjonen.

`HttpSession` API - Timeout

- Vi bør alltid ha en fornuftig timeout på tjener-sesjonen ved inaktivitet. Dette sikrer at brukere som glemmer å logge ut og stenge nettleseren likevel får en viss beskyttelse. Dette gjør også at vi får ryddet opp litt på tjeneren, og at vi ikke får et økende antall "gamle" sesjoner som må holdes i minnet.

- API-kall:

```
sesjon.setMaxInactiveInterval(<sekunder>)
```

Scopes

Scope = Omfang / Rekkevidde / Gyldighetsområde / Levetid ?

Begrepet brukes i mange sammenhenger, og det er en del nyanser. F.eks.

- En variabel kan ha et blokk-/metode-scope, det vil si at variabelen kun er **gyldig** innenfor blokken/metoden. Dette handler om kildekode og hvordan programmet skal forstås.
- Data kan ha request-/session-scope, dvs. at data kun er tilgjengelig innenfor samme request/session. Vi har sett at **parametre og model-attributter har request-scope**, mens **sesjon-attributter har session-scope**. Dette handler om runtime, livssyklus og **levetid** for data.
- Når vi snakker om det videre i dag er det i denne betydningen. Vi kaller det Data-Scopes.

PS! Neste gang skal vi også se at Spring-komponenter (Spring Beans) har livssyklus og levetid, så da får vi en tredje betydning av scope.)

Ulike Data-Scopes i web-applikasjoner

- Loop / Iteration Scope: `th:each=«p : ...» og deretter ${p}`
  Altså at p kun er tilgjengelig mens løkken utføres
- Request / Model Scope: `model.addAttribute(«p», ...) og deretter ${p}`
  Altså at p kun er tilgjengelig i denne requesten
- Flash Scope: `redirectAttributes.addFlashAttribute(«p», ...) og deretter ${p}`
  Altså at p kun er tilgjengelig i requesten etter redirecten
- Session Scope: `session.setAttribute(«p», ...) og deretter ${session.p}`
  Altså at p kun er tilgjengelig i denne sesjonen (for denne brukeren)
- Application / Context Scope: `??? og deretter ${application.p}`
  Altså at p kun er tilgjengelig i denne applikasjonen
  NB! Må sørge for trådsikkerhet hvis data deles av flere brukere !!!

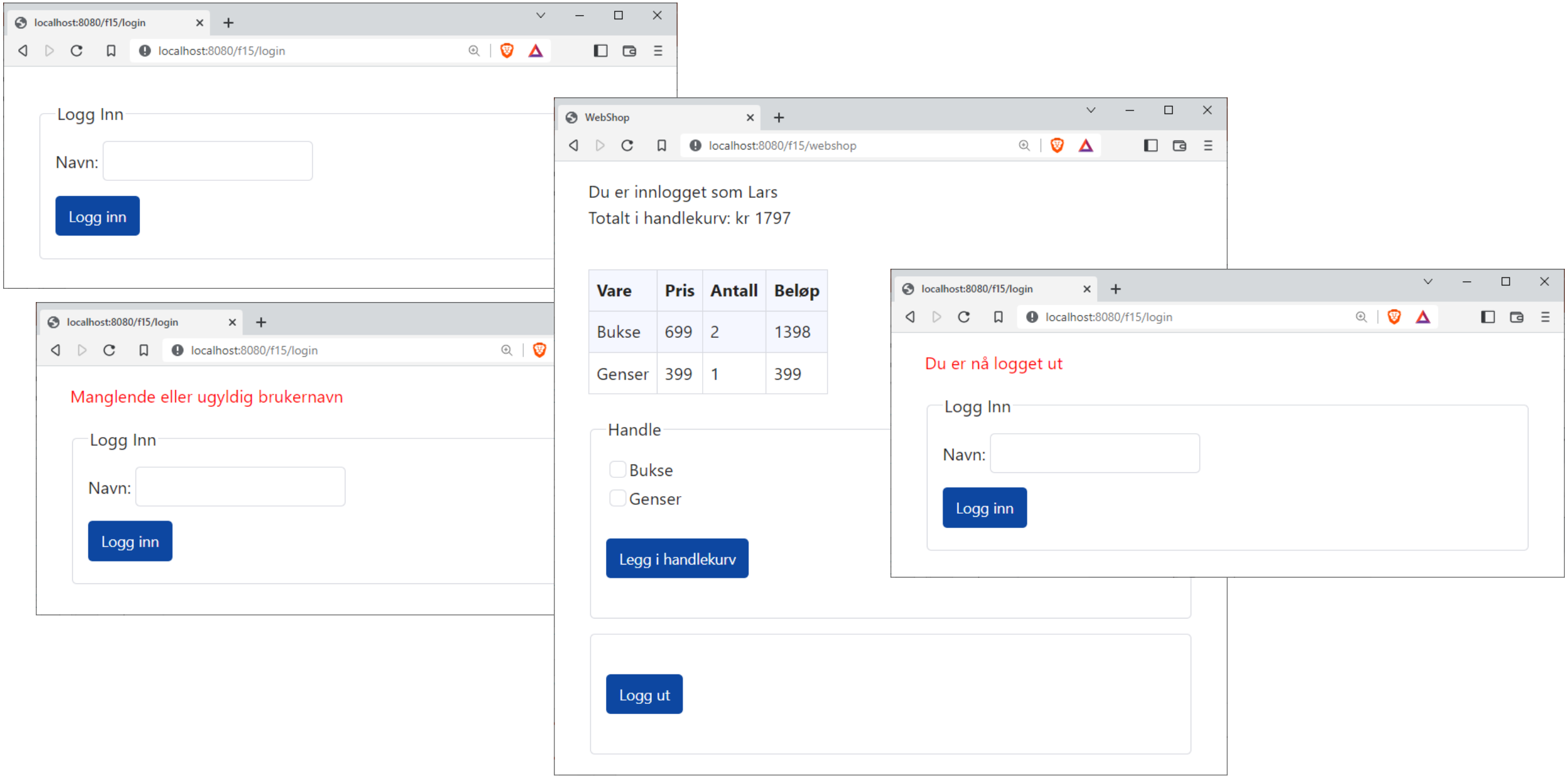
Pause



Eksempel webshop-app

På tide å se på et konkret eksempel. Vi tar en enkel handlevogn-applikasjon som også krever en slags innlogging.




localhost:8080/f15/login
Logg Inn
Navn:
Logg inn
WebShop
localhost:8080/f15/webshop
Du er innlogget som Lars
Totalt i handlekurv: kr 1797
Vare
Pris
Antall
Beløp
Bukse
699
2
1398
Genser
399
1
399
Handle
Bukse
Genser
Legg i handlekurv
Logg ut
Manglende eller ugyldig brukernavn
Du er nå logget ut

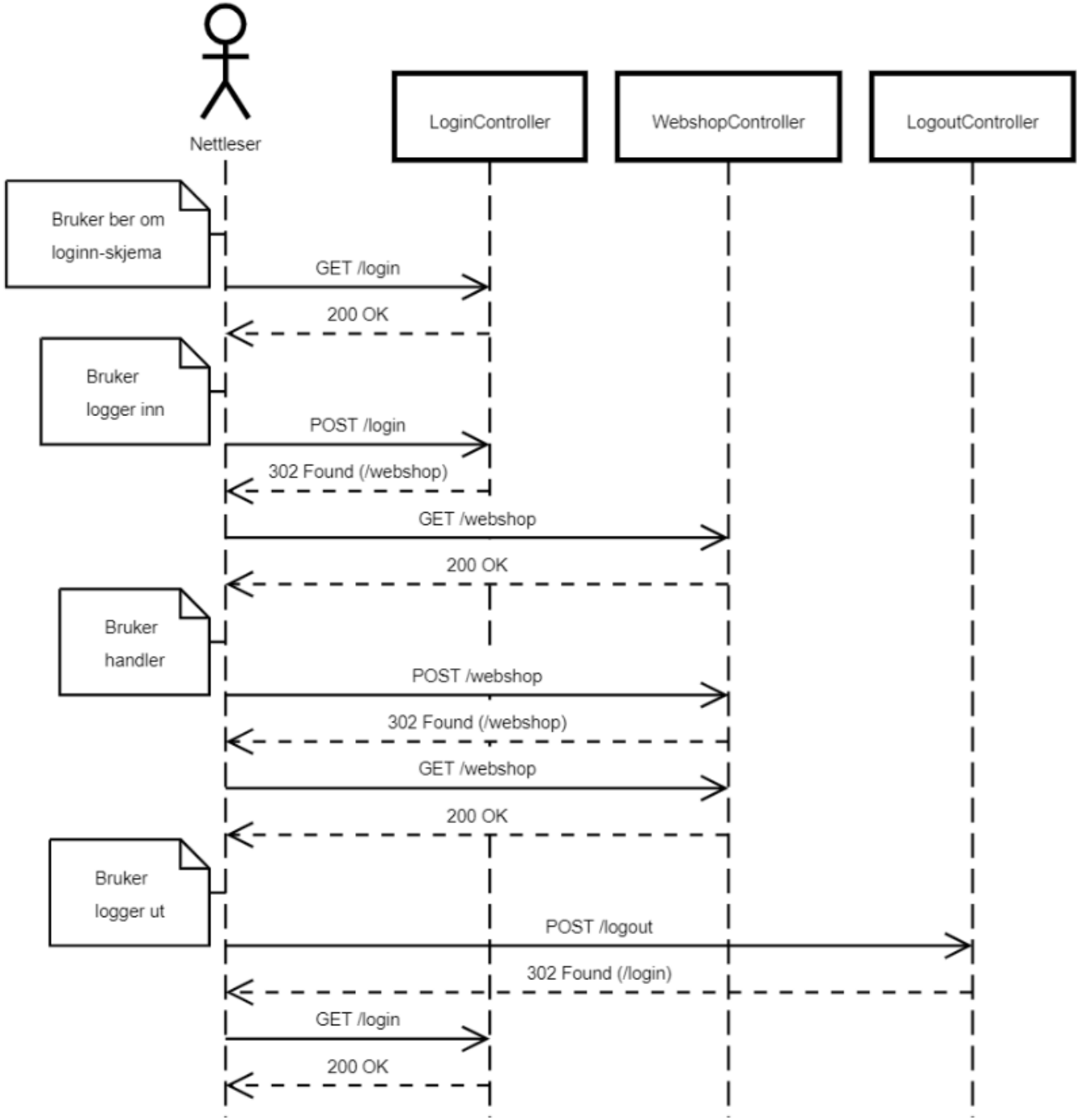
Oversikt over brukstilfeller og requests

- GET /login -> Vise innloggingssiden (->View)
- POST /login -> Logge inn. Deretter gå til butikksiden (PRG)

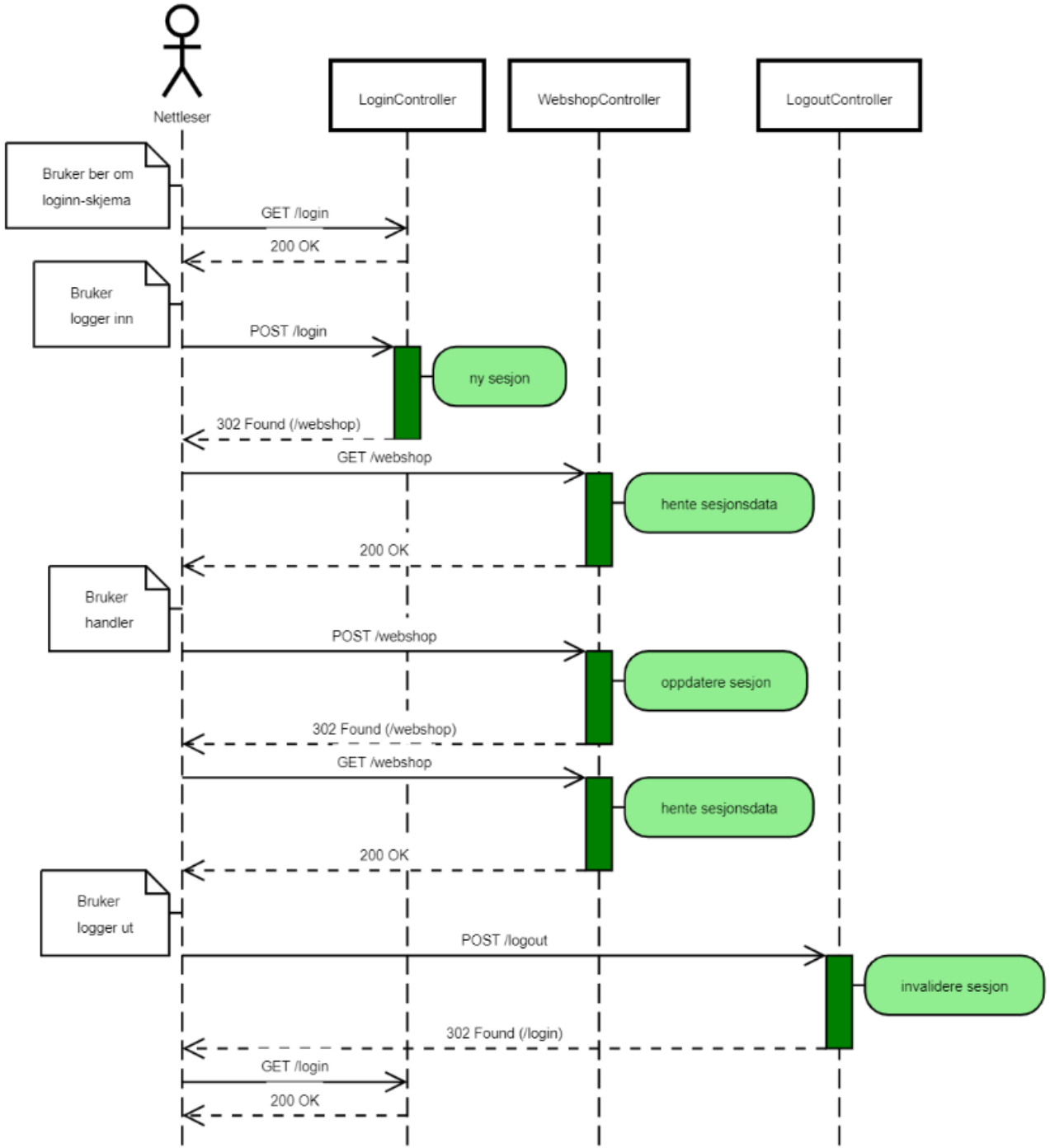
- GET /webshop -> Vise butikksiden (->View)
- POST /webshop -> Legge varer i kurven. Deretter gå til butikksiden igjen (PRG)

- POST /logout -> Logge ut. Deretter gå til innloggingssiden (PRG)

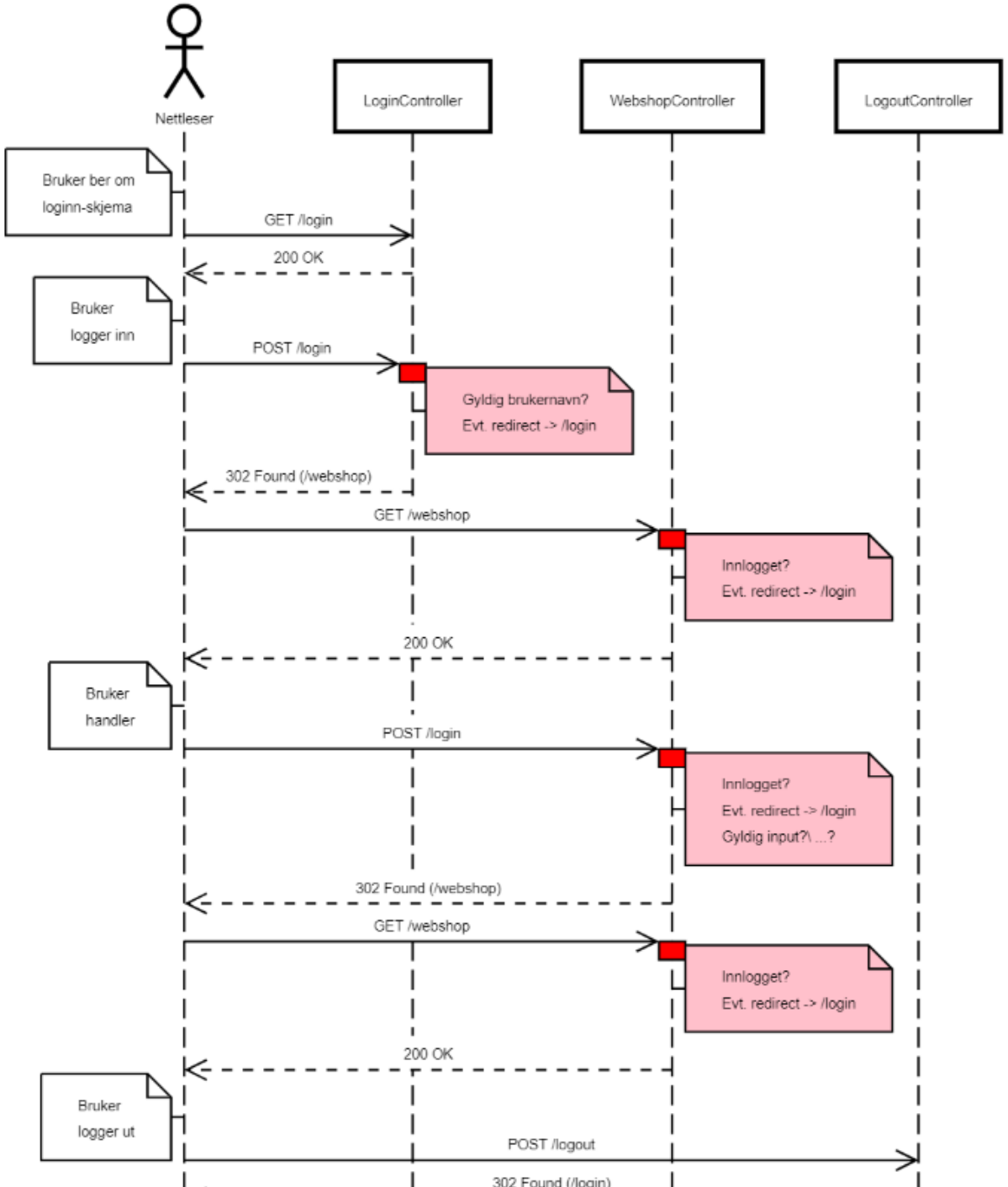
"Happy path"


Nettleser
LoginController
WebshopController
LogoutController
Bruker ber om loginn-skjema
GET /login
200 OK
Bruker logger inn
POST /login
302 Found (/webshop)
GET /webshop
200 OK
Bruker handler
POST /webshop
302 Found (/webshop)
GET /webshop
200 OK
Bruker logger ut
POST /logout
302 Found (/login)
GET /login
200 OK

Sesjonsdata



Feilhåndtering



Eksempel – Web-butikk

Prosjekt: 📁 f19b-sesjoner-innlogging



- Har laget et skjelett som jeg vil bruke som utgangspunkt.
- Så gjør vi sammen ...

Nettleser-sesjon versus tjener-sesjon

- Når vi jobber med web-applikasjoner må vi alltid huske på at vi har 2 datamaskiner som inngår: klient og tjener. Av og til snakker de med hverandre, men stort sett lever de hver for seg.
- En **nettleser-sesjon** varer så lenge man har åpent et nettleservindu på klienten. Levetiden til sesjons-cookies følger levetiden til nettleser-sesjonen.
- En **tjener-sesjon** varer så lenge vi bestemmer oss for, enten til vi "invaliderer" den eller den "går ut på tid". Tjeneren har ingen formening om nettleseren hos klienten kjører eller ikke.

Nettleser-sesjon versus tjener-sesjon

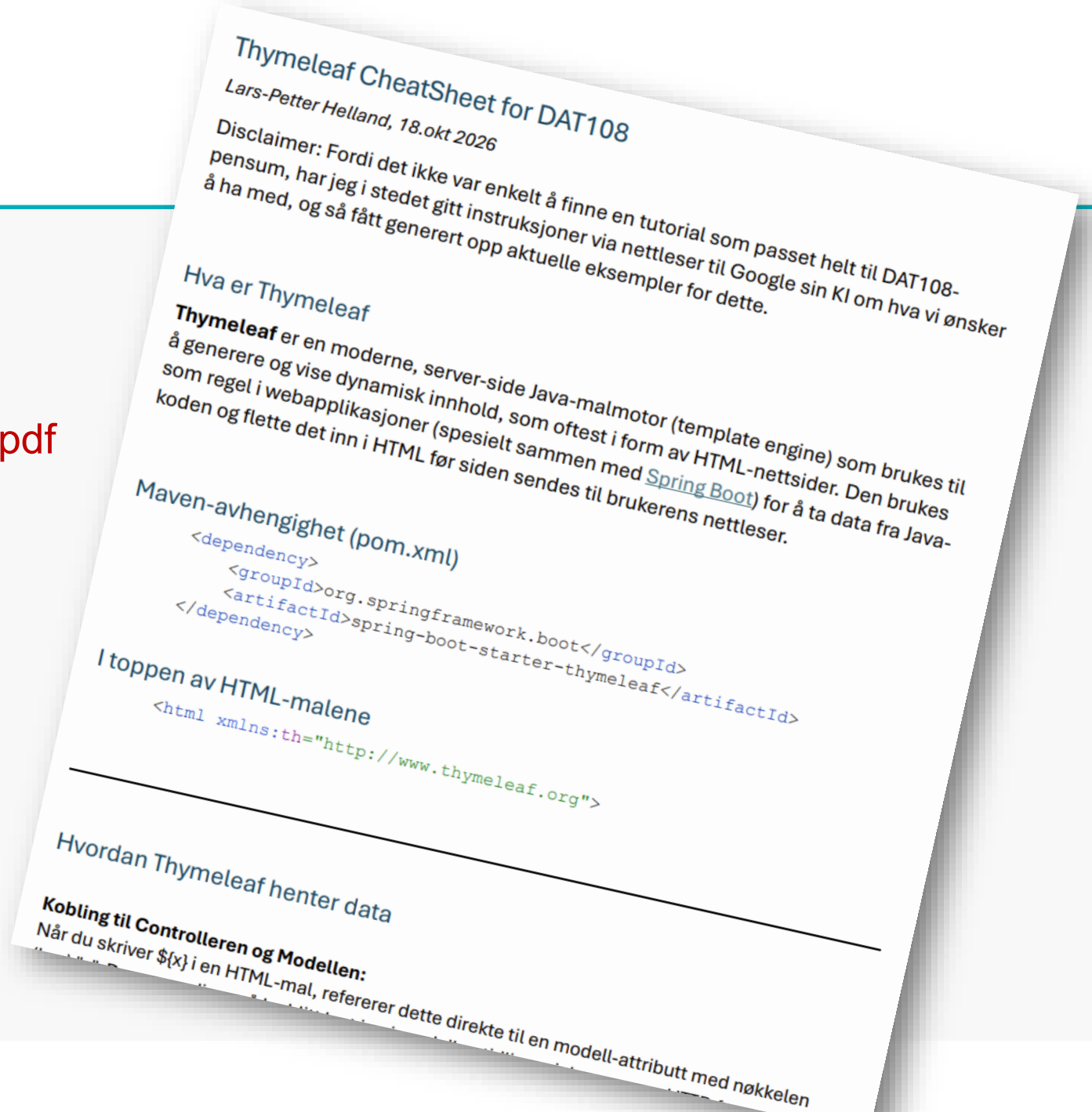
- Vi bør alltid gi brukeren mulighet til å aktivt "invalidere" tjener-sesjonen, dvs. få "logge ut". Dette sikrer mot at neste bruker på samme maskin får tilgang til innlogget innhold hos forrige bruker.
- Vi bør alltid oppmuntre til å stenge nettleseren etter utlogging. Dette sikrer at sesjons-cookien i nettleseren "dør" og at sesjonsdataene da uansett blir utilgjengelige.

Thymeleaf !

Nå er det lagt ut en ny utgave av Thymeleaf-kompendiet som er pensum.

Thymeleaf Jukselapp for DAT108.pdf

(5 sider)



Thymeleaf CheatSheet for DAT108

Lars-Petter Helland, 18.okt 2026

Disclaimer: Fordi det ikke var enkelt å finne en tutorial som passet helt til DAT108-pensum, har jeg i stedet gitt instruksjoner via nettleser til Google sin KI om hva vi ønsker å ha med, og så fått generert opp aktuelle eksempler for dette.

Hva er Thymeleaf

Thymeleaf er en moderne, server-side Java-malmotor (template engine) som brukes til å generere og vise dynamisk innhold, som oftest i form av HTML-nettsider. Den brukes som regel i webapplikasjoner (spesielt sammen med Spring Boot) for å ta data fra Java-koden og flette det inn i HTML før siden sendes til brukerens nettleser.

Maven-avhengighet (pom.xml)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

I toppen av HTML-malene

```
<html xmlns:th="http://www.thymeleaf.org">
```

Hvordan Thymeleaf henter data

Kobling til Controlleren og Modellen:
Når du skriver ${x} i en HTML-mal, refererer dette direkte til en modell-attributt med nøkkelen

Videre

- Oblig4 Frist søndag 25. oktober
- Fredag: Om komponenter i Spring, om lagdeling, og om oppkobling til database
- **Forberedelse: Finn frem og test at databasekontoen du hadde i DAT107 fremdeles fungerer**